

Project Proposal: DC Motor Speed Control System

Yuqi Li - s336721

November 2025

1 Introduction

This project describes the design of an MCU-centric cascade closed-loop DC motor speed control system. The user sets a target speed via a potentiometer, and an OLED screen displays both the target and actual speeds in real-time.

The system architecture is centered around an STM32 microcontroller, which executes the complete **Sensing-Decision-Making- Control** loop. The MCU reads user input and motor feedback (both encoder and actual current), runs a cascade PID control algorithm with an outer speed loop and an inner current loop, and sends the calculated PWM control word to an FPGA via the SPI bus. The FPGA, acting as an SPI slave, then generates precise PWM waveforms according to these commands to drive the motor.

2 Hardware Architecture

All MCU peripherals (ADC, Timers, SPI, I2C) will be configured and controlled by directly accessing their registers, without using any HAL libraries or drivers.

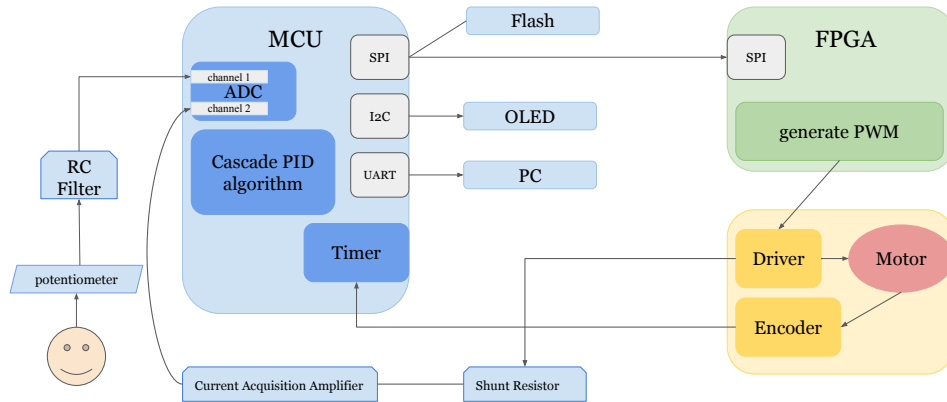


Figure 1: Architecture

2.1 MCU

1. The internal ADC is used for two primary inputs:

- a. it reads the potentiometer, A hardware RC low-pass filter is placed before the ADC pin, and a moving average filter is implemented in software.

- b. it reads the motor's actual current from a current sense amplifier for the inner control loop.
2. A timer is used to receive pulses from the motor's encoder, counting them to allow the main program to calculate the actual speed.
 3. The cascade PID closed-loop control algorithm is executed periodically. The outer loop (speed loop) calculates a desired current based on the error between desired and actual speeds. The inner loop (current loop) then uses this desired current and the actual measured current to calculate the final PWM duty cycle value.
 4. It acts as the SPI bus master, communicating with the FPGA and an external Flash memory chip.
 5. It drives the OLED screen via the I2C bus.
 6. It connects to a computer via a UART interface to print debugging information for the developer (e.g., real-time PID error, output values).
 7. The potentiometer is connected to an MCU pin.

2.2 FPGA

1. Design and implement an SPI slave receiver to parse commands and data sent by the MCU.
2. Design and implement a high-resolution digital PWM waveform generator. This module will continuously generate PWM waves with the corresponding duty cycle based on the value received via SPI.

2.3 Motor Drive Unit

1. The motor's encoder output is connected directly to the MCU's timer pins.
2. The motor driver IC receives the PWM signal from the FPGA and drives the motor.
3. A Shunt Resistor is placed in the motor's power path (H-bridge) to measure the current.
4. A current sense amplifier reads the very small voltage drop across the shunt resistor and amplifies this voltage signal to a level that the MCU's ADC can accurately read..

2.4 External Memory

An SPI NOR Flash chip is connected to the MCU for non-volatile storage of system configuration parameters, such as PID coefficients and user calibration data.

2.5 Power Management

1. The PWM signal generated by the FPGA is used to modulate the switches in the H-bridge, thereby controlling the power delivered to the motor.
2. On the power supply side, to prevent electrical noise from the motor from affecting the ADC sampling, a separate 3.3V logic supply for the MCU and FPGA is designed using the noise suppression characteristics of a Linear Dropout Regulator (LDO).

3 System Workflow

1. **Sensing:** All sensing tasks are performed by the MCU. The ADC module reads the hardware-filtered potentiometer voltage and applies a software filter to obtain a stable target speed. Concurrently, the timer's encoder mode provides hardware decoding of encoder pulses, and the MCU reads the count to get the actual speed. In parallel, a second ADC channel continuously samples the actual motor current via the sense amplifier.

2. **Decision-Making:** All algorithms run on the MCU, now implemented as a cascade PID controller. The outer speed loop compares the target speed and actual speed to calculate a desired current reference. This reference is then fed to the inner current loop, which compares the desired current with the actual measured current to generate the final PWM duty cycle value.
3. **Command Transmission:** The MCU sends the calculated PWM value to the FPGA via the SPI bus.
4. **Hardware Execution:** The FPGA's SPI slave module receives this value and updates it in the PWM generator's duty cycle register. The parallel PWM generator module immediately begins producing a PWM waveform with the corresponding duty cycle.
5. **Drive & Display:** The motor driver IC drives the motor. The MCU displays relevant data on the OLED screen.